NUTRICARE

Guia tecnica para comprender, explicar y defender el proyecto

React Native, Expo, TypeScript y Figma aplicados a una experiencia multiplataforma para pacientes y nutricionistas.

APRENDE	EXPLICA	DEMUESTRA
Arquitectura, carpetas, rutas, estado, validacion y persistencia.	Guion tecnico de 8 a 10 minutos y ruta de demostracion.	Evidencias concretas para responder la rubrica estrategica.

Objetivo de aprendizaje

Al terminar esta guia podras describir como funciona NutriCare, abrir el proyecto en Visual Studio Code, recuperar el servidor local en el puerto 8082, explicar las decisiones de UX/UI, demostrar las funciones principales y publicar el codigo en GitHub con seguridad.

Preparado para el avance multiplataforma y la presentacion tecnica del proyecto.

MODULO 01

Como usar esta guia en menos de 24 horas

Una ruta intensiva, realista y enfocada en comprender en lugar de memorizar.

Bloque	Duracion	Actividad	Resultado
1. Mapa mental	90 min	Lee modulos 2 a 6. Abre las carpetas mientras lees.	Puedes narrar la arquitectura sin mirar notas.
2. Flujo real	2 h	Sigue un registro de paciente y una fotografia desde pantalla hasta almacenamiento.	Entiendes datos, validacion y persistencia.
3. Diseño	90 min	Compara tokens, componentes y pantallas con los principios de Figma.	Justificas consistencia y reutilizacion.
4. Operacion	90 min	Practica localhost 8082, VS Code y GitHub.	Recuperas y publicas el proyecto.
5. Presentacion	2 h	Ensaya el guion dos veces y cronometra la demostracion.	Video claro de 13 a 15 min.
6. Repaso	45 min	Responde las preguntas del modulo final sin leer.	Seguridad para preguntas del docente.

Regla de oro: no intentes aprender cada linea. Aprende primero **responsabilidades**: que carpeta muestra pantallas, cual guarda datos, cual valida y cual mantiene la apariencia.

Checkpoint

[ ] Puedo explicar NutriCare en 30 segundos. [ ] Puedo nombrar las cinco capas del proyecto. [ ] Puedo mostrar un flujo completo de usuario.

Explicacion de 30 segundos sugerida

NutriCare es una aplicacion multiplataforma construida con React Native y Expo. Separa la interfaz, navegacion, estado, validacion y datos de demostracion. Maneja dos roles, nutricionista y paciente; reutiliza componentes visuales; valida formularios; y conserva informacion localmente para demostrar flujos completos sin depender de un servidor externo.

MODULO 02

La rubrica convertida en evidencias

Que debes mostrar para justificar el nivel estrategico.

Criterio de la captura	Evidencia en NutriCare	Como demostrarlo
Pantallas profesionales y coherentes con Figma	Paleta, espaciado, radios y sombras centralizados en theme.ts; BrandHeader y componentes UI reutilizables.	Abre Inicio, Plan, Perfil y un formulario. Señala que la misma identidad se mantiene.
Componentes reutilizables	Screen, SurfaceCard, FormField, PrimaryButton, BrandHeader y BirthDatePicker.	Cambia de pantalla y muestra patrones repetidos sin duplicar estilos.
Buenas practicas multiplataforma	React Native, Expo Router, SafeArea, ScrollView, Platform y componentes accesibles.	Ejecuta web y explica que la base tambien apunta a Android/iOS.
Funcionalidad completa y robusta	Autenticacion por rol, pacientes, agenda, medidas, fotos, comentarios, checklist, persistencia y validacion.	Haz un flujo de alta, guarda, recarga y comprueba permanencia.
Teoria y practica integradas	Las decisiones se conectan con documentacion oficial de React Native, Expo y Figma.	Usa los modulos de investigacion y la bibliografia APA 7.

Frase de defensa

No solo se diseñaron pantallas: se convirtio un sistema visual de Figma en tokens y componentes reutilizables, y se completo un flujo de datos demostrable con validacion y persistencia local.

MODULO 03

Que problema resuelve NutriCare

Dos experiencias conectadas por el mismo modelo de seguimiento nutricional.

Rol	Necesidad	Funciones principales
Nutricionista	Organizar seguimiento y tomar decisiones con informacion ordenada.	Resumen, agenda, pacientes, citas, mediciones, planes, fotografias y comentarios.
Paciente	Entender su plan y registrar evidencia cotidiana.	Plan del dia, checklist, progreso, fotos de comida, comentarios recibidos y perfil.

El proyecto usa cuentas de demostracion para que el evaluador pruebe ambos recorridos. La navegacion cambia segun el rol: el nutricionista ve Pacientes y Medicion; el paciente ve Progreso. Esta decision evita mostrar acciones irrelevantes y reduce carga cognitiva.

Recorrido recomendado

- Inicia como nutricionista y enseña agenda, nuevo paciente, nueva consulta y medicion.
- Cierra sesion e inicia como paciente para enseñar plan, checklist, progreso y fotografias.
- En ambos roles abre Perfil para evidenciar reutilizacion y edicion de datos.

Idea clave

Una sola base de codigo puede ofrecer experiencias distintas. El rol no crea dos aplicaciones: configura rutas, etiquetas y acciones dentro de la misma arquitectura.

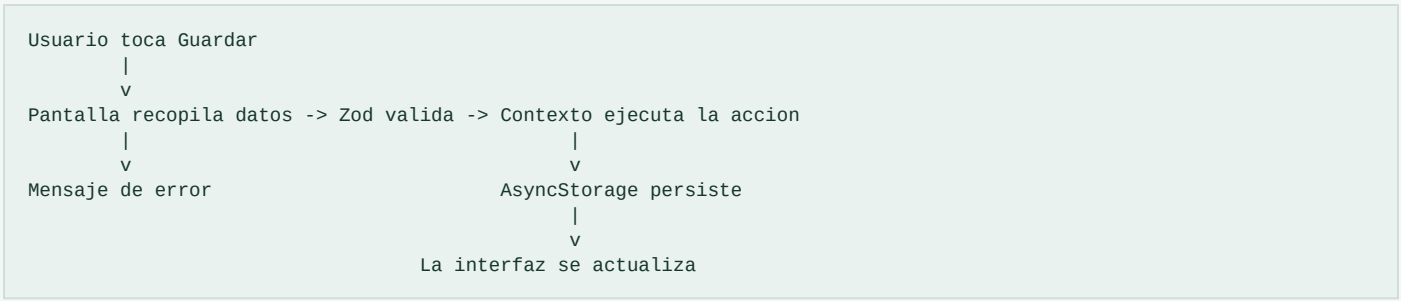
MODULO 04

Arquitectura: el mapa completo

Piensa en capas conectadas, no en archivos aislados.

1. PANTALLAS	2. COMPONENTES UI	3. CONTEXTOS
src/app Rutas, formularios, tableros y navegacion.	src/components/ui Piezas visuales repetibles.	src/context Estado y operaciones por dominio.
4. ESQUEMAS	**5. DATOS Y TIPOS**	**6. ALMACENAMIENTO**
src/schemas Reglas Zod para entradas validas.	src/data + src/types Datos demo y contratos TypeScript.	AsyncStorage Persistencia local entre recargas.

Flujo mental



Expo Router conecta los archivos de **src/app** con URLs y pantallas. Los contextos evitan pasar datos manualmente por muchos niveles. Los esquemas bloquean entradas incorrectas y AsyncStorage conserva el resultado local.

MODULO 05

Distribucion de carpetas, explicada

Esta es la parte que debes poder señalar en Visual Studio Code.

```
nutricare/
|-- mobile/                 Aplicacion Expo / React Native
|   |-- assets/             Logos, iconos e imagenes estaticas
|   |-- src/
|   |   |-- app/            Pantallas y rutas de Expo Router
|   |   |   |-- (auth)/     Login, registro y recuperacion
|   |   |   |-- (tabs)/     Navegacion principal por pestañas
|   |   |   |-- *.tsx       Flujos secundarios y formularios
|   |   |-- components/ui/  Componentes visuales reutilizables
|   |   |-- constants/      Tokens del tema (color, espacio, radio)
|   |   |-- context/        Estado global y persistencia por dominio
|   |   |-- data/           Contenido de demostracion
|   |   |-- schemas/        Validaciones con Zod
|   |   `-- types/          Tipos TypeScript compartidos
|   |-- app.json            Configuracion de Expo, permisos y web
|   `-- package.json        Dependencias y comandos de la app movil
|-- public/                 Recursos de la version web raiz
|-- src/                    Aplicacion web raiz / soporte del proyecto
|-- README.md               Puerta de entrada para ejecutar el repositorio
|-- pnpm-workspace.yaml     Define paquetes del workspace
`-- pnpm-lock.yaml          Versiones exactas de dependencias
```

Por que src/app es especial

Expo Router usa enrutamiento basado en archivos: la ubicacion y el nombre del archivo participan en la ruta. Las carpetas entre parentesis, como **(auth)** y **(tabs)**, agrupan pantallas sin agregar ese nombre a la URL. Los archivos **_layout.tsx** definen navegadores y proveedores compartidos.

Prueba rapida

En VS Code presiona Ctrl+P y abre, en este orden: mobile/src/app/_layout.tsx, mobile/src/app/(tabs)/_layout.tsx, mobile/src/context/AuthContext.tsx y mobile/src/constants/theme.ts.

MODULO 06

Rutas, layouts y navegacion por rol

Como una estructura de archivos se convierte en una aplicacion navegable.

El archivo raiz **mobile/src/app/_layout.tsx** monta los proveedores globales y declara las rutas principales. Esto hace que cualquier pantalla pueda acceder a sesion, pacientes, mediciones, fotos y citas.

```
SafeAreaProvider
`-- AuthProvider
    `-- PatientProvider
        `-- MeasurementProvider
            `-- PhotoProvider
                `-- AppointmentProvider
                    `-- Stack de Expo Router
```

El archivo **mobile/src/app/(tabs)/_layout.tsx** decide que pestañas se ven. Las pestañas comunes son Inicio, Planes y Perfil. Para nutricionista agrega Pacientes y Medicion; para paciente agrega Progreso.

Accion	Herramienta	Significado
Ir a otra pantalla	router.push('/ruta')	Conserva historial y permite volver.
Volver	router.back()	Regresa a la pantalla anterior.
Reemplazar	router.replace('/ruta')	Evita volver a una pantalla que ya no corresponde, por ejemplo login.
Agrupar rutas	(auth), (tabs)	Organiza el codigo sin ensuciar la URL.

Buena practica

La navegacion no debe contener toda la logica del negocio. Su responsabilidad es mostrar destinos y mover al usuario; los contextos se encargan de datos y acciones.

MODULO 07

Componentes reutilizables y sistema visual

La conexion directa entre Figma y el codigo.

Componente	Responsabilidad	Beneficio
BrandHeader	Logo circular, nombre y distintivo del rol.	Identidad consistente en todas las pantallas.
Screen	Area segura, fondo, scroll y espaciado base.	Evita repetir estructura y problemas de pantalla.
SurfaceCard	Tarjetas con borde, radio y sombra.	Jerarquia visual uniforme.
FormField	Etiqueta, entrada, estado y error.	Formularios coherentes y accesibles.
PrimaryButton	Accion principal con estados.	Misma prioridad visual en todo el flujo.
BirthDatePicker	Seleccion de año, mes y dia.	Patron reutilizable en alta y perfil.

theme.ts funciona como la traduccion de estilos/variables de Figma al codigo. En vez de inventar un verde o un radio en cada pantalla, se consumen tokens compartidos: colores, espaciado, radios y sombras.

```
Figma                        Codigo
Color / variable verde  --->  colors.primary
Espaciado 16            --->  spacing.md
Radio de tarjeta         --->  radii.card
Componente de boton      --->  <PrimaryButton />
Auto layout              --->  flexDirection, gap, padding, flex
```

Justificacion

La reutilizacion reduce divergencias visuales, acelera cambios y hace que la implementacion sea verificable frente al diseño. Si cambia un token, el ajuste se propaga a los consumidores correctos.

MODULO 08

Estado global y persistencia local

Que ocurre cuando guardas y luego recargas.

React muestra una interfaz a partir del **estado**. En NutriCare, cada dominio tiene un contexto: AuthContext, PatientContext, MeasurementContext, PhotoContext y AppointmentContext. Un contexto expone datos y funciones para que las pantallas no dupliquen reglas.

Contexto	Datos que administra	Ejemplo de accion
AuthContext	Sesion, cuentas y perfil editado.	login, register, updateProfile, logout
PatientContext	Pacientes creados.	addPatient
MeasurementContext	Mediciones y medidas adicionales.	addMeasurement
PhotoContext	Fotografias, comida y comentarios.	savePhoto, deletePhoto
AppointmentContext	Consultas con fecha, hora y motivo.	addAppointment

AsyncStorage es un almacenamiento persistente de pares clave-valor. En este proyecto sirve para una demostracion sin backend: al guardar un dato se actualiza el estado y tambien se serializa. Al iniciar la app, el contexto hidrata su estado leyendo la clave correspondiente.

Limitacion importante

AsyncStorage es local y no cifrado. Es correcto para prototipo y datos demo; no es una base de datos clinica ni el lugar apropiado para contraseñas o expedientes reales. La version final debe usar API, autenticacion segura y almacenamiento protegido.

MODULO 09

Ejemplo completo: guardar un paciente

Aprende este flujo y podras explicar casi todos los formularios.

Paso	Que ocurre	Archivo o capa
1	El usuario escribe nombre, correo, fecha, sexo y objetivo.	new-patient.tsx
2	React Hook Form administra valores, errores y envio.	Pantalla / formulario
3	Zod comprueba formato y reglas, incluida la edad adulta.	schemas/patient.ts
4	La pantalla llama a addPatient con datos validos.	PatientContext.tsx
5	El contexto agrega el registro y lo guarda en AsyncStorage.	Estado + persistencia
6	Aparece confirmacion y la lista refleja el nuevo paciente.	Interfaz

```
Entrada del usuario
  -> control del formulario
  -> esquema Zod
      -> error visible, si falla
      -> accion del contexto, si pasa
          -> estado actualizado
          -> AsyncStorage actualizado
          -> confirmacion al usuario
```

Este patrón también se reconoce en nueva consulta, registro de medicion, registro/edicion de perfil y fotografias. La diferencia esta en los campos y el contexto utilizado, no en la arquitectura.

Pregunta de defensa

¿Por que validar antes del contexto? Porque la capa de entrada puede dar retroalimentacion inmediata y el dominio recibe datos con una forma confiable.

MODULO 10

Validacion, formularios y TypeScript

Tres capas que reducen errores antes de guardar.

TypeScript comprueba durante el desarrollo que los objetos tengan propiedades y tipos esperados. **Zod** valida los datos cuando la app esta ejecutandose. **React Hook Form** coordina la interaccion del formulario. Ninguno reemplaza completamente a los otros.

Herramienta	Momento	Que evita
TypeScript	Al escribir/compilar	Usar una propiedad inexistente o un tipo equivocado.
Zod	Al enviar datos	Correos invalidos, campos vacios, fechas no permitidas o valores fuera de rango.
React Hook Form	Durante la interaccion	Manejo manual repetitivo de valores, errores y envio.

Ejemplos del proyecto

- La fecha de nacimiento debe representar una persona mayor de edad.
- El correo debe tener formato valido y los campos requeridos no pueden quedar vacios.
- Las mediciones aceptan unidades y valores consistentes; las medidas adicionales siguen una estructura tipada.
- Los errores se muestran cerca del campo para que la persona sepa que corregir.

Explicacion simple

TypeScript cuida al programador; Zod cuida el dato real; React Hook Form cuida la experiencia al capturarlo.

MODULO 11

Funciones del nutricionista

Que demostrar y que argumento tecnico acompaña cada pantalla.

Pantalla	Accion demostrable	Argumento tecnico
Inicio	Resumen, agenda con fecha y acceso a nueva consulta.	Composicion de tarjetas y datos de AppointmentContext.
Pacientes	Lista, alta de paciente y acceso a mediciones.	Validacion, persistencia y navegacion a flujos secundarios.
Nueva consulta	Fecha, hora AM/PM, paciente y motivo.	Selector reutilizable, opciones controladas y confirmacion.
Medicion	Datos principales y medidas adicionales.	Coleccion dinamica validada por esquema.
Planes	Plan mensual y detalle diario.	Datos estructurados y componentes repetibles.
Fotografias	Revisar evidencia y comentarios.	Contexto compartido para seguimiento visual.

Secuencia sugerida en video

- Abre Inicio y relaciona tarjetas con jerarquia visual.
- Crea una consulta para mañana y usa la seleccion de hora AM/PM.
- Crea o abre un paciente, registra una medicion y añade una medida adicional.
- Muestra que los mensajes de confirmacion cierran el ciclo de accion.

MODULO 12

Funciones del paciente

Seguimiento diario, comprension y retroalimentacion.

Pantalla	Accion demostrable	Argumento tecnico
Inicio	Peso, proxima cita y plan del dia.	Informacion priorizada por contexto de uso.
Plan de hoy	Marcar circulos como checklist.	Estado interactivo que contribuye a la meta semanal.
Planes	Consultar menu y abrir detalle.	Patron de lista a detalle.
Fotografias	Elegir galeria/camara, tipo de comida y guardar.	Permisos de Expo ImagePicker y persistencia local.
Guardadas	Ver comentarios y borrar fotografia.	Retroalimentacion, modal de confirmacion y operacion CRUD.
Perfil	Editar nombre y fecha de nacimiento.	Componente compartido y validacion de edad.

Por que el checklist importa

Los circulos ya no son decorativos: cada toque representa una comida completada. Ese dato hace coherente la meta semanal y transforma el plan en una herramienta de seguimiento.

En la demostracion marca dos comidas, recarga y comprueba si el estado esperado permanece. Explica que una version conectada enviaria el avance al backend para que el nutricionista lo consulte desde otro dispositivo.

MODULO 13

Fotografias, camara y comportamiento multiplataforma

El mismo objetivo de usuario necesita adaptaciones por plataforma.

Expo ImagePicker permite seleccionar imagenes de la galeria o abrir la camara. La aplicacion solicita permiso cuando corresponde y mantiene un flujo alternativo. En web, la eleccion debe nacer de una accion directa del usuario; en dispositivos, el sistema operativo administra el permiso y la interfaz nativa.

Caso	Comportamiento	Razon
Galeria	Seleccionar archivo/imagen existente.	Flujo rapido y compatible con web.
Camara	Solicitar permiso y capturar fotografia.	Respeta la privacidad y capacidades del dispositivo.
Guardar	Asociar tipo de comida y comentario opcional.	Convierte una imagen en informacion util.
Eliminar	Confirmar antes de borrar.	Evita perdida accidental.
Comentario profesional	Mostrar retroalimentacion en la vista guardada.	Cierra el ciclo paciente-nutricionista.

Seguridad

Las fotos de salud pueden ser sensibles. En produccion se requieren consentimiento, autenticacion, almacenamiento privado, cifrado y reglas de retencion; base64/AsyncStorage solo es aceptable para demostracion local.

MODULO 14

Investigacion: React Native aplicado a NutriCare

Sintesis teorica conectada con decisiones visibles en el codigo.

React Native propone construir interfaces con componentes de React que se traducen a vistas nativas. Componentes basicos como View, Text, Image, TextInput, Pressable y ScrollView forman la base de la interfaz. En NutriCare, las pantallas combinan estos bloques y los encapsulan en componentes propios para conservar consistencia (Meta Platforms, Inc., s. f.-a; Meta Platforms, Inc., s. f.-b).

Expo Router agrega navegacion basada en archivos para Android, iOS y web. La regla de mantener pantallas en **src/app** y componentes no navegables fuera de esa carpeta coincide con la organizacion del proyecto (Expo, s. f.-a; Expo, s. f.-b).

La aplicacion usa una estrategia de estado por dominios con Context, validacion con Zod y persistencia con AsyncStorage. Esto mantiene separadas las responsabilidades, aunque el almacenamiento local no sustituye una API segura. AsyncStorage se describe oficialmente como persistente, asincrono y no cifrado, por lo que su uso aqui se limita a la demostracion (React Native Async Storage, s. f.).

Principio	Aplicacion en NutriCare	Efecto
Composicion	Pantallas formadas por componentes UI propios y nativos.	Menos duplicacion.
Un solo proyecto	Rutas y estilos compartidos para web y plataformas moviles.	Mayor alcance con una base comun.
Adaptacion	Permisos, SafeArea y decisiones dependientes de plataforma.	Experiencia apropiada por dispositivo.
Estado declarativo	La UI refleja sesion, listas, fotos y checklist.	Cambios predecibles.

MODULO 15

Investigacion: Figma aplicado a NutriCare

Del archivo visual a un sistema mantenible.

Figma permite definir estilos, componentes, variables y auto layout. Los estilos reutilizan propiedades visuales; los componentes actuan como bloques repetibles; las variables almacenan valores reutilizables y modos; y auto layout administra espaciado, relleno, alineacion y cambios de contenido (Figma, s. f.-a, s. f.-b, s. f.-c, s. f.-d).

Concepto Figma	Traduccion al proyecto	Decision justificable
Estilos y variables	constants/theme.ts	Una fuente de verdad para color, espacio, radio y sombra.
Componentes	components/ui	Botones, campos, encabezados y tarjetas reutilizados.
Auto layout	Flexbox, gap, padding y contenedores responsivos.	El contenido se adapta sin posiciones rigidas.
Variantes/estados	Activo, inactivo, error, seleccionado y completado.	La interfaz comunica el estado de la accion.
Prototipo	Rutas y flujos implementados con Expo Router.	La navegacion deja de ser simulada y se vuelve funcional.

Idea para defender

Figma no se uso como una imagen para copiar pixel por pixel. Se uso como especificacion de un sistema: tokens, componentes, estados y relaciones que luego se implementaron con reglas equivalentes en React Native.

MODULO 16

Decisiones de diseño y buenas practicas

Como justificar lo que se ve y lo que no se ve.

Decision	Justificacion UX/UI	Justificacion tecnica
Verde oscuro + esmeralda + menta	Asocia bienestar, mantiene contraste y crea jerarquia.	Tokens centralizados permiten cambios globales.
Logo dentro de circulo	Separa el fondo del logo y limpia cambios de superficie.	BrandHeader evita implementaciones diferentes.
Tarjetas redondeadas	Agrupan informacion y hacen escaneable el tablero.	SurfaceCard encapsula radio, borde y sombra.
Confirmaciones	El usuario sabe que guardar/eliminar tuvo resultado.	Modales o alertas cierran el ciclo de estado.
Controles seleccionables	Calendarios, listas y AM/PM reducen errores de formato.	Datos controlados facilitan validacion.
Iconos con etiqueta	La etiqueta elimina ambiguedad visual.	Accesibilidad y navegacion mas clara.

Accesibilidad a mencionar

- Contraste entre texto y superficies; el color no debe ser la unica señal.
- Etiquetas visibles, mensajes de error cercanos y areas tactiles suficientemente grandes.
- accessibilityRole, accessibilityLabel y accessibilityState en controles interactivos.
- Scroll y SafeArea para evitar contenido cortado en pantallas pequeñas.

MODULO 17

Que esta completo y que falta para produccion

La honestidad tecnica fortalece la presentacion.

Nivel actual: prototipo funcional	Siguiente nivel: producto
Flujos navegables por rol	Autenticacion con tokens seguros y recuperacion real
Persistencia local con AsyncStorage	API + base de datos + sincronizacion multiusuario
Datos y cuentas demo	Usuarios reales, permisos y control de acceso
Fotos locales	Almacenamiento privado en nube y politicas de retencion
Validacion cliente	Validacion duplicada en servidor y auditoria
Pruebas manuales	Pruebas unitarias, integracion y E2E automatizadas
Ejecucion web y base Expo	Pruebas en dispositivos Android/iOS y despliegue

Como decirlo

El avance supera una maqueta porque los flujos guardan y recuperan datos. Aun asi, no afirmamos que sea un sistema clinico listo para produccion: faltan backend, seguridad, sincronizacion y pruebas automatizadas.

Esta diferenciacion demuestra criterio profesional: un prototipo valida experiencia y arquitectura; un producto agrega infraestructura, seguridad, monitoreo y cumplimiento.

MODULO 18

Comandos principales del proyecto

Una hoja de referencia para trabajar desde PowerShell.

Comando	Para que sirve
pnpm install	Instala las dependencias declaradas en el workspace.
pnpm --filter mobile start	Inicia Expo para elegir plataforma.
pnpm --filter mobile web	Abre la aplicacion movil en navegador.
pnpm --filter mobile lint	Ejecuta revision estatica del codigo movil.
pnpm --filter mobile exec tsc --noEmit	Comprueba tipos sin generar archivos.
git status	Muestra cambios y archivos sin seguimiento.
git diff --stat	Resume cuanto cambio antes de confirmar.
git log --oneline -5	Muestra los cinco commits mas recientes.

Comandos basicos de PowerShell

```
Get-Location                # carpeta actual
Get-ChildItem               # contenido de la carpeta
Set-Location .\mobile       # entrar a mobile
Set-Location ..             # subir una carpeta
code .                      # abrir la carpeta actual en VS Code
```

Cuidado

Ejecuta los comandos desde la raiz nutricare, salvo cuando se indique mobile. No borres node_modules, .git o archivos de configuracion si no sabes por que. Antes de actualizar dependencias, crea un commit recuperable.

MODULO 19

Restaurar localhost en el puerto 8082

Procedimiento seguro cuando Expo no abre o el puerto quedo ocupado.

Opcion A: arranque normal desde la raiz

```
Set-Location "C:\Users\Fatima Oviedo\OneDrive - Universidad Don Bosco\Escritorio\nutricare"
pnpm install
$env:EXPO_HOME = (Join-Path (Resolve-Path '.\mobile').Path '.expo-home')
$env:EXPO_NO_TELEMETRY = "1"
$env:BROWSER = "none"
node .\mobile\node_modules\expo\bin\cli start --web --port 8082
```

Cuando la terminal muestre que Metro esta listo, abre **http://localhost:8082**. Mantener esa terminal abierta es obligatorio; cerrarla detiene el servidor.

Opcion B: desde la carpeta mobile

```
Set-Location .\mobile
$env:EXPO_HOME = (Join-Path (Resolve-Path '.').Path '.expo-home')
$env:EXPO_NO_TELEMETRY = "1"
$env:BROWSER = "none"
node .\node_modules\expo\bin\cli start --web --port 8082
```

Si aparece otro puerto

Expo puede proponer 8083 si 8082 esta ocupado. Puedes aceptar temporalmente o liberar 8082 con el diagnostico de la pagina siguiente.

MODULO 20

Diagnostico de localhost

Sigue el orden y detente cuando funcione.

Sintoma	Comprobacion	Solucion
No reconoce pnpm	pnpm --version	Instala/habilita pnpm o usa el Node configurado para el proyecto.
Faltan modulos	Revisa node_modules y el error	Desde la raiz ejecuta pnpm install.
Puerto ocupado	Get-NetTCPConnection -LocalPort 8082	Identifica el proceso antes de detenerlo.
Pantalla desactualizada	Recarga dura del navegador	Ctrl+Shift+R; reinicia Expo limpiando cache.
Metro falla	Lee la primera linea roja	Corrige la ruta/archivo indicado; no ignores el primer error.

Liberar 8082 con verificacion

```
$connection8082 = Get-NetTCPConnection -LocalPort 8082 -State Listen -ErrorAction SilentlyContinue
$connection8082 | Select-Object LocalAddress, LocalPort, OwningProcess
Get-Process -Id $connection8082.OwningProcess

# Solo si confirmaste que es un servidor anterior de Expo/Node:
Stop-Process -Id $connection8082.OwningProcess
```

Reinicio con cache limpia

```
node .\mobile\node_modules\expo\bin\cli start --web --port 8082 --clear
```

Seguridad

No detengas un proceso solo por su numero. Primero usa Get-Process y confirma que corresponde a Node/Expo de este proyecto.

MODULO 21

Ruta de explicacion en Visual Studio Code

Que archivos abrir y que debes decir en cada uno.

Orden	Archivo	Explicacion de 1 frase
1	mobile/package.json	Dependencias y comandos que construyen/ejecutan la app.
2	mobile/app.json	Configuracion Expo, permisos, iconos y salida web.
3	mobile/src/app/_layout.tsx	Proveedores globales y rutas de alto nivel.
4	mobile/src/app/(tabs)/_layout.tsx	Pestañas visibles segun el rol.
5	mobile/src/constants/theme.ts	Tokens que traducen el sistema visual.
6	mobile/src/components/ui	Piezas reutilizables de la interfaz.
7	mobile/src/context	Estado, operaciones y persistencia por dominio.
8	mobile/src/schemas	Reglas de validacion de los formularios.
9	mobile/src/types	Contratos de datos compartidos.
10	mobile/src/data	Datos demo separados de la presentacion.

Consejo de grabacion: aumenta el zoom del editor a 125-140 %, oculta paneles innecesarios y abre antes los archivos. No navegues buscando durante el video.

Checkpoint

[ ] El explorador muestra mobile/src completo. [ ] La terminal esta en 8082. [ ] No se ven contraseñas personales ni archivos .env. [ ] Las pestañas estan preparadas en el orden anterior.

MODULO 22

Subir el proyecto a GitHub

Flujo conservador para no publicar basura ni secretos.

El repositorio ya apunta a **https://github.com/fatimayoviedo/Nutricare_ProyectoDPS.git**. Ejecuta desde la raiz.

```
git status
git remote -v
git diff --stat
```

1. Revisa .gitignore y confirma que excluye node_modules, .expo-home, archivos temporales y secretos. 2. Agrega solo lo que corresponde. 3. Revisa de nuevo antes del commit.

```
git add mobile README.md package.json pnpm-lock.yaml pnpm-workspace.yaml tsconfig.json .gitignore

# Si el PDF forma parte de la entrega:
git add output/pdf/Guia_tecnica_NutriCare_React_Native_Figma.pdf

git status
git diff --cached --stat
git commit -m "feat: completar avance movil y documentacion tecnica"
```

Sincroniza y publica

```
git pull --rebase origin main
git push origin main
```

Si hay conflicto

No uses git reset --hard. Abre los archivos marcados, conserva el contenido correcto, ejecuta git add y luego git rebase --continue. Si dudas, cancela con git rebase --abort y pide revision.

Antes de compartir el enlace, abre GitHub y confirma que el ultimo commit aparece, que README explica como ejecutar y que no se publicaron .env, credenciales reales, node_modules ni carpetas temporales.

MODULO 23

Plan del video

8 a 10 minutos de explicacion tecnica, mas demostracion visual.

Tiempo	Bloque	Visual
0:00-0:40	Problema, usuarios y alcance	Diapositivas 1-2
0:40-1:30	Tecnologias	Diapositiva 3
1:30-2:40	Arquitectura por capas	Diapositiva 4
2:40-3:50	Carpetas y Expo Router	Diapositivas 5-6 + VS Code
3:50-5:00	Componentes y Figma	Diapositivas 7-8
5:00-6:15	Estado, validacion y persistencia	Diapositivas 9-10
6:15-7:15	Multiplataforma y decisiones	Diapositiva 11
7:15-8:20	Robustez, limites y siguiente paso	Diapositivas 12-13
8:20-9:00	Cierre tecnico	Diapositiva 14
9:00-15:00	Demostracion libre de pantallas	App en localhost:8082

Regla de ritmo

No leas la diapositiva. Cada una debe darte palabras clave; la explicacion completa esta en el guion. Ensaya para que el bloque tecnico quede entre 8 y 10 minutos.

MODULO 24

Guion del video - Parte 1

Apertura, tecnologias y arquitectura. Duracion aproximada: 4 minutos.

0:00-0:40 - Apertura

Hola. En este video presento NutriCare, una aplicacion multiplataforma orientada al seguimiento nutricional. El proyecto atiende dos perfiles: el nutricionista, que organiza pacientes, consultas, mediciones y retroalimentacion; y el paciente, que consulta su plan, registra evidencia y observa su progreso. El objetivo no fue crear solo pantallas, sino implementar flujos coherentes, reutilizables y demostrables.

0:40-1:30 - Tecnologias

La interfaz esta construida con React Native y Expo. React Native permite componer pantallas con componentes y compartir una base entre plataformas. Expo facilita el entorno, los permisos y la ejecucion web. Expo Router organiza la navegacion a partir de archivos. TypeScript define la forma esperada de los datos; React Hook Form administra los formularios; Zod valida la informacion en tiempo de ejecucion; y AsyncStorage conserva datos localmente durante la demostracion.

1:30-2:40 - Arquitectura

La arquitectura se puede leer en capas. Primero estan las pantallas de src/app. Luego los componentes de src/components/ui, como campos, botones, tarjetas y encabezados. Los contextos de src/context concentran el estado y las acciones por dominio. Los esquemas de src/schemas validan formularios. Los tipos definen contratos y data contiene contenido de demostracion. Finalmente, AsyncStorage persiste el estado local. Cuando el usuario guarda, la pantalla valida, el contexto actualiza y persiste, y la interfaz refleja el resultado.

2:40-4:00 - Carpetas y rutas

En Visual Studio Code, mobile es la aplicacion Expo. Dentro de src/app, las carpetas auth y tabs agrupan los flujos. Los archivos layout definen los navegadores y los proveedores compartidos. La navegacion cambia por rol: ambos perfiles comparten Inicio, Planes y Perfil; nutricionista agrega Pacientes y Medicion, mientras paciente agrega Progreso. Los componentes no navegables se mantienen fuera de app, lo que sigue la organizacion recomendada por Expo Router.

MODULO 25

Guion del video - Parte 2

Figma, datos, robustez y cierre. Duracion aproximada: 5 minutos.

4:00-5:00 - Figma y componentes

La relacion con Figma se implemento como sistema, no como una captura copiada. Los colores, espacios, radios y sombras se traducen a tokens en theme.ts. Los elementos repetidos se convierten en componentes como BrandHeader, SurfaceCard, FormField y PrimaryButton. El auto layout de Figma se corresponde con Flexbox, gap y padding. Esto conserva la identidad visual, evita duplicacion y permite cambios globales controlados.

5:00-6:15 - Formularios y persistencia

Un ejemplo es Nuevo paciente. React Hook Form recopila los valores; Zod comprueba correo, campos obligatorios y edad; PatientContext agrega el registro y AsyncStorage lo conserva. La confirmacion informa que la operacion termino. El mismo patron se reutiliza en consultas, mediciones, perfil y fotografias. TypeScript protege al desarrollador, Zod protege el dato real y React Hook Form protege la experiencia de captura.

6:15-7:15 - Multiplataforma

La base se ejecuta en web y esta preparada para Android e iOS con Expo. Aun compartiendo codigo, se respetan diferencias: SafeArea evita zonas del sistema, ScrollView responde a pantallas pequeñas, Platform ajusta estilos y Expo ImagePicker gestiona galeria, camara y permisos. Esto es una estrategia multiplataforma: reutilizar lo comun y adaptar lo que depende del dispositivo.

7:15-8:20 - Robustez y limites

El avance supera una maqueta porque permite autenticarse por rol, crear pacientes y citas, registrar mediciones, seleccionar fechas y horas, guardar y eliminar fotos, ver comentarios, marcar comidas como checklist y conservar informacion local. Para produccion aun se requiere backend, autenticacion segura, base de datos, almacenamiento privado de fotos y pruebas automatizadas. AsyncStorage es intencional para el prototipo, no para expedientes clinicos reales.

8:20-9:00 - Cierre tecnico

En sintesis, NutriCare integra teoria y practica: Figma define un sistema visual coherente; React Native lo convierte en componentes; Expo Router organiza la experiencia; y los contextos, esquemas y almacenamiento completan los flujos. A continuacion mostrare las pantallas de ambos roles y comprobare que las acciones principales funcionan.

MODULO 26

Demostracion de pantallas y control de calidad

Usa esta lista despues del guion tecnico.

Orden	Demostracion	Evidencia
1	Login de nutricionista	Rol y pestañas correctas.
2	Nueva consulta para otra fecha	Calendario, hora AM/PM, guardado y agenda con fecha.
3	Nuevo paciente	Calendario de nacimiento, validacion y popup.
4	Nueva medicion	Acceso desde Medicion y lista de medidas adicionales.
5	Login de paciente	Navegacion especifica del rol.
6	Plan de hoy	Circulos seleccionables y meta semanal coherente.
7	Fotografia	Galeria/camara, espacio entre botones, guardado, comentario y borrado.
8	Perfil	Edicion y selector de fecha reutilizado.

Checklist antes de grabar

- [ ] Localhost abre exactamente en http://localhost:8082.
- [ ] Las cuentas demo funcionan y conoces la contraseña que vas a mostrar.
- [ ] No hay notificaciones, datos personales ni pestañas irrelevantes visibles.
- [ ] El navegador esta en un ancho que muestre claramente el diseño movil.
- [ ] El cursor y el zoom permiten ver cada accion.
- [ ] Hiciste una prueba completa sin cortes y dura 13-15 minutos.

MODULO 27

Preguntas frecuentes para la defensa

Respuestas cortas que demuestran criterio tecnico.

Pregunta	Respuesta sugerida
¿Por que React Native?	Permite reutilizar conceptos y gran parte del codigo entre plataformas, conservando componentes cercanos a la experiencia nativa.
¿Por que Expo?	Reduce configuracion inicial y aporta router, permisos y herramientas para ejecutar y probar rapidamente.
¿Por que Context y no una libreria mayor?	El alcance actual es manejable y los dominios estan separados. Si aumenta la complejidad, se evaluaria una solucion especializada.
¿AsyncStorage es una base de datos?	No. Es almacenamiento local clave-valor, asincrono y no cifrado; aqui habilita la demostracion.
¿Como se conecto Figma con el codigo?	Variables/estilos se volvieron tokens, componentes se volvieron componentes UI y auto layout se implemento con Flexbox.
¿Que hace robusto el avance?	Los recorridos validan, guardan, confirman, recuperan y permiten corregir/eliminar; no son pantallas estaticas.
¿Que falta?	Backend, autenticacion segura, nube para fotos, sincronizacion, pruebas automatizadas y validacion en dispositivos reales.

MODULO 28

Glosario express

Terminos suficientes para hablar con precision.

Termino	Definicion sencilla
Componente	Pieza reutilizable de interfaz con propiedades y comportamiento.
Prop	Dato que un componente padre entrega a un componente hijo.
Estado	Informacion que puede cambiar y provoca una nueva representacion de la UI.
Context	Mecanismo para compartir datos/acciones sin pasarlos manualmente por cada nivel.
Hook	Funcion de React que conecta un componente con estado, efectos u otras capacidades.
Ruta	Destino navegable de la aplicacion.
Esquema	Regla ejecutable que decide si un dato tiene forma y valores validos.
Token	Nombre reutilizable para una decision visual, como un color o espaciado.
Persistencia	Capacidad de conservar informacion despues de cerrar o recargar.
Backend	Servicio del lado servidor que aplica reglas, autentica y almacena datos.
CRUD	Crear, leer, actualizar y eliminar datos.
Repositorio	Historial versionado del proyecto administrado con Git.

Ultimo reto

Sin leer: explica el recorrido Guardar fotografia usando al menos cinco terminos del glosario. Si puedes hacerlo en un minuto, ya entiendes la arquitectura esencial.

MODULO 29

Referencias en formato APA 7

Fuentes oficiales para ampliar y verificar la investigacion.

Expo. (s. f.-a). *Expo Router*. Expo Documentation. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/versions/latest/sdk/router/

Expo. (s. f.-b). *Core concepts*. Expo Documentation. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/router/basics/core-concepts/

Expo. (s. f.-c). *Navigation*. Expo Documentation. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/router/basics/navigation/

Expo. (s. f.-d). *ImagePicker*. Expo Documentation. Recuperado el 18 de octubre de 2026, de https://docs.expo.dev/versions/latest/sdk/imagepicker/

Figma. (s. f.-a). *Guide to styles in Figma*. Figma Learn. Recuperado el 18 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360039238753-Guide-to-styles-in-Figma

Figma. (s. f.-b). *Lesson 3: Build your design system*. Figma Learn. Recuperado el 18 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/14548865734679-Lesson-3-Build-your-design-system

Figma. (s. f.-c). *Overview of variables, collections, and modes*. Figma Learn. Recuperado el 18 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/14506821864087-Overview-of-variables-collections-and-modes

Figma. (s. f.-d). *Guide to auto layout in Figma*. Figma Learn. Recuperado el 18 de octubre de 2026, de https://help.figma.com/hc/en-us/articles/360040451373-Guide-to-auto-layout-in-Figma

MODULO 30

Referencias APA 7 - Continuacion

Bibliografia tecnica complementaria.

Meta Platforms, Inc. (s. f.-a). *Core components and native components*. React Native. Recuperado el 18 de octubre de 2026, de https://reactnative.dev/docs/next/intro-react-native-components

Meta Platforms, Inc. (s. f.-b). *Core components and APIs*. React Native. Recuperado el 18 de octubre de 2026, de https://reactnative.dev/docs/components-and-apis

React Hook Form. (s. f.). *Get started*. Recuperado el 18 de octubre de 2026, de https://react-hook-form.com/get-started

React Native Async Storage. (s. f.). *Async Storage*. Recuperado el 18 de octubre de 2026, de https://react-native-async-storage.github.io/

The TypeScript Team. (s. f.). *The TypeScript handbook*. TypeScript. Recuperado el 18 de octubre de 2026, de https://www.typescriptlang.org/docs/handbook/intro.html

Zod. (s. f.). *TypeScript-first schema validation with static type inference*. Recuperado el 18 de octubre de 2026, de https://zod.dev/

Nota de investigacion

Se priorizaron documentos oficiales para describir capacidades y conceptos. Las afirmaciones sobre la arquitectura de NutriCare se verificaron directamente en el codigo local del proyecto.

Cierre

Ya no necesitas memorizar el proyecto como una lista de pantallas. Puedes explicarlo como un sistema: Figma define decisiones visuales; los componentes las reutilizan; las rutas organizan recorridos; los contextos administran acciones; los esquemas protegen datos; y la persistencia conserva la demostracion.